THE LAZY EVALUATOR OF HENDERSON AND MORRIS, BY INTERSECTION TYPES

ABSTRACT. Henderson and Morris [1] give a lazy evaluator for a pure LISP as seven rewrite rules on a memory, and sketch a proof that the rules preserve a Scott–Strachey semantics, the delicate case being the rule that creates a circular structure. We present the semantics by *intersection types*: the compact elements of the value domain are the types, ordered by a subtyping relation given by inference rules, and the meaning of a memory cell is the set of types derivable for it. All the relations are inductive and positive. There is no approximation index, no computed join and no consistency hypothesis in the invariance proofs. The circular rule is proved by one induction on derivations, and partial correctness of the evaluator follows. This is the informal basis for a formalisation in Unit with indexed inductive families. Finally, the infinite elements are the filters of types. The semantic function of the paper is then the least solution of its own equations, and the invariance theorem holds in the paper's form, as equalities of semantic memories, each rule being a short calculation and the circular rule two applications of fixed-point induction.

1. The language and the memory

We follow §II of the paper. Expressions are

$$e \;::=\; \xi \mid (\mathsf{QUOTE}\ a) \mid (\mathsf{CONS}\ e\ e) \mid (\mathsf{CAR}\ e) \mid (\mathsf{CDR}\ e) \mid (\mathsf{ATOM}\ e) \mid (\mathsf{EQ}\ e\ e)$$
$$\mid (\mathsf{IF}\ e\ e\ e) \mid (e\ e) \mid (\mathsf{LAMBDA}\ \xi\ e) \mid (\mathsf{LABEL}\ \xi\ e) \mid (\mathsf{FUNARG}\ e\ \theta)$$

with ξ a variable, a an atom and $\theta = \xi_1\, e_1 \ldots \xi_n\, e_n$ an association list. A *memory* μ maps addresses π to *cells*: the same forms with addresses in place of subexpressions. Two memories differ *at* π_0 if they agree everywhere except at π_0; $\mu[e/\pi_0]$ is μ with the cell e at π_0.

The rules are those of the paper: $[C]$ ($\mathsf{CAR}/\mathsf{CDR}$ of a CONS), $[A]$, $[E]$, $[I]$, $[F]$ (a: variable lookup in a FUNARG, the smallest i with $\xi_i = \xi$; b: a quoted atom; c: an application under a FUNARG; d: a primitive under a FUNARG), $[G]$ (applying a FUNARG-closed LAMBDA) and $[L]$:

$$\mu(\pi_0) = (\mathsf{FUNARG}\ \pi_\ell\ \theta), \quad \mu(\pi_\ell) = (\mathsf{LABEL}\ \xi\ \pi_1) \implies \mu' = \mu[(\mathsf{FUNARG}\ \pi_1\ \xi\ \pi_0\ \theta)/\pi_0],$$

which creates a circular structure. Each rule rewrites one cell; $[Fc]$ and $[Fd]$ first allocate new cells. An allocated cell is *fresh*: free, and occurring in no cell of the memory. In the paper this is part of the consistency of the memory ("no non-free cell points at a free one"), and it is the only consistency property used below. The evaluator Eval of the paper performs a sequence of such rewrites.

2. Types

The value domain of the paper is $V = A + (V \times V) + (V \to V)$, with Reynolds's separated sum [6], so that $\bot$, "some atom", $(\mathsf{CONS}\ \bot\ \bot)$ and $(\mathsf{LAMBDA}\ x\ \bot)$ are distinct. Its compact elements are described by the *types*

$$\sigma, \tau \;::=\; \omega \mid \mathsf{atom} \mid \ulcorner a \urcorner \mid \sigma \times \tau \mid \sigma \to \tau \mid \sigma \wedge \tau.$$

Here ω is no information, atom is "an atom", $\ulcorner a \urcorner$ the atom a, $\sigma \times \tau$ a pair whose components have types σ and τ, and $\sigma \to \tau$ a function sending σ to τ. Thus $\omega \times \omega$ is "a pair" and $\omega \to \omega$ "a function". A finite step function $\{(u_1, v_1), \ldots, (u_n, v_n)\}$ is the type $(u_1 \to v_1) \wedge \cdots \wedge (u_n \to v_n)$.

Definition 2.1 (subtyping)**.** $\sigma \le \tau$ is the least relation closed under: reflexivity and transitivity; $\sigma \wedge \tau \le \sigma$, $\sigma \wedge \tau \le \tau$, and $\rho \le \sigma,\ \rho \le \tau \ \Rightarrow\ \rho \le \sigma \wedge \tau$; $\sigma \le \omega$; $\ulcorner a \urcorner \le \mathsf{atom}$; $\sigma \le \sigma',\ \tau \le \tau' \Rightarrow \sigma \times \tau \le \sigma' \times \tau'$ and $(\sigma \times \tau) \wedge (\sigma' \times \tau') \le (\sigma \wedge \sigma') \times (\tau \wedge \tau')$; $\sigma' \le \sigma,\ \tau \le \tau' \Rightarrow \sigma \to \tau \le \sigma' \to \tau'$, $(\sigma \to \tau) \wedge (\sigma \to \tau') \le \sigma \to (\tau \wedge \tau')$, and $\sigma \to \omega \le \omega \to \omega$.

The last axiom says that every arrow type gives "a function": all the types $\sigma \to \omega$ are equivalent to $\omega \to \omega$. It replaces the axiom $\omega \le \omega \to \omega$ of the usual systems, which would identify "a function" with "no information" and so contradict the separated sum.

Presenting compact elements as types is the domain interpretation of Martin-Löf [2], also the basis of Abramsky's domain theory in logical form [3]; for the untyped λ-calculus it is the filter model [4]. The order is given by inference rules, so the join of step functions never has to be computed: it is assembled in a derivation from $\wedge$ and the arrow rules, and the characterisation of the order becomes a lemma. Write a type as a meet of *components* (types that are not meets). A *selection* of components is a finite multiset of them, possibly empty; the meet of the empty selection is ω.

Lemma 2.2 (inversion of subtyping)**.** *Let ρ have components ρ_k.*

1. *$\rho \le \ulcorner a \urcorner$ iff some ρ_k is $\ulcorner a \urcorner$; $\rho \le \mathsf{atom}$ iff some ρ_k is atom or an $\ulcorner a \urcorner$.*
2. *$\rho \le \sigma \times \tau$ iff ρ has a pair component and a selection of pair components $\sigma_i \times \tau_i$ with $\bigwedge \sigma_i \le \sigma$ and $\bigwedge \tau_i \le \tau$.*
3. *(β-soundness) $\rho \le \sigma \to \tau$ iff ρ has an arrow component and a selection of arrow components $\sigma_i \to \tau_i$ with $\sigma \le \bigwedge \sigma_i$ and $\bigwedge \tau_i \le \tau$.*

Proof sketch. The right-hand sides define a relation $\le^s$, *syntax directed*: inductive on the target and positive (the order appears only in premises). It contains the generating rules and is reflexive. Transitivity of $\le^s$ is proved by induction on the depth of the middle type, meets not counted: in the arrow case, $\rho \le^s \pi$ and $\pi \le^s \sigma \to \tau$ give a selection of arrows $\pi_k \to \pi'_k$ of π, each inverted to a selection of arrows of ρ; their union (a meet of selections, which is why selections are multisets) works, using transitivity through the meets of the π_k and of the π'_k, whose depth is smaller than that of π (their size need not be, a selection may repeat a component). Hence $\le = \le^s$. □

Types may be inconsistent ($\ulcorner a \urcorner \wedge \ulcorner b \urcorner$ with $a \ne b$, or $\mathsf{atom} \wedge (\omega \times \omega)$). The elements of V are the consistent filters of types; nothing below needs consistency.

3. The meaning of a memory cell

An *environment* Γ assigns types to finitely many variables; $\Gamma(\xi) = \omega$ otherwise, and $\Gamma[\xi : \sigma]$ updates it. For an association list $\theta = \xi_1 \pi_1 \dots \xi_n \pi_n$ and types $\vec{\sigma}$, write $\Gamma[\theta : \vec{\sigma}] = \Gamma[\xi_n : \sigma_n] \cdots [\xi_1 : \sigma_1]$, so that the smallest i wins, as in rule $[Fa]$.

Definition 3.1 (observation)**.** $\mu \vdash \pi :_\Gamma \sigma$ ("in μ, the cell π has type σ in Γ") is the least relation closed under the rules $\mu \vdash \pi :_\Gamma \omega$; $\mu \vdash \pi :_\Gamma \sigma,\ \mu \vdash \pi :_\Gamma \tau \Rightarrow \mu \vdash \pi :_\Gamma \sigma \wedge \tau$; $\mu \vdash \pi :_\Gamma \sigma,\ \sigma \le \tau \Rightarrow \mu \vdash \pi :_\Gamma \tau$; and, according to the cell $\mu(\pi)$:

- ξ: $\mu \vdash \pi :_\Gamma \Gamma(\xi)$; (QUOTE a): $\mu \vdash \pi :_\Gamma \ulcorner a \urcorner$;
- (CONS $\pi_1\ \pi_2$): from $\mu \vdash \pi_1 :_\Gamma \sigma$ and $\mu \vdash \pi_2 :_\Gamma \tau$, the type $\sigma \times \tau$;
- (CAR π'): from $\mu \vdash \pi' :_\Gamma \sigma \times \tau$, the type σ; CDR likewise with τ;
- (ATOM π'): from atom, the type $\ulcorner \mathsf{T} \urcorner$; from $\omega \times \omega$, the type $\ulcorner \mathsf{NIL} \urcorner$;
- (EQ $\pi_1\ \pi_2$): from $\ulcorner a_1 \urcorner$ and $\ulcorner a_2 \urcorner$, the type $\ulcorner \mathsf{T} \urcorner$ if $a_1 = a_2$ and $\ulcorner \mathsf{NIL} \urcorner$ otherwise;
- (IF $\pi_0\ \pi_1\ \pi_2$): from $\ulcorner \mathsf{T} \urcorner$ at π_0 and σ at π_1, the type σ; from $\ulcorner \mathsf{NIL} \urcorner$ at π_0 and σ at π_2, the type σ;

- $(\pi_0\ \pi_1)$: from $\sigma \to \tau$ at π_0 and σ at π_1, the type τ;
- (LAMBDA $\xi\ \pi'$): from $\mu \vdash \pi' :_{\Gamma[\xi:\sigma]} \tau$, the type $\sigma \to \tau$;
- (LABEL $\xi\ \pi'$): from $\mu \vdash \pi' :_{\Gamma[\xi:\sigma]} \tau$ and $\mu \vdash \pi :_\Gamma \sigma$, the type τ;
- (FUNARG $\pi'\ \theta$): from $\mu \vdash \pi_i :_\Gamma \sigma_i$ for each binding $\xi_i\pi_i$ of θ and $\mu \vdash \pi' :_{\Gamma[\theta:\vec{\sigma}]} \tau$, the type τ.

This is the paper's $V\mu\pi\rho$ read as an inductive relation: the meaning of π in an environment is the filter of its types (Section 6 makes this precise). The paper argues with the approximations V_i of V and Y_i of the fixed point; here the height of derivations plays their role. Being inductive, the relation is a least fixed point, so the LABEL clause *is* the paper's $Y\{\lambda x.\ldots\}$; and a circular memory simply has no derivations that would need infinite unfolding. Note that the LABEL rule has a premise about its own address π. Environments are lists of types, not of values. The first three rules make the set of types of a cell a filter. Two basic facts:

Lemma 3.2. (1) *(environments) If $\Delta(\xi) \le \Gamma(\xi)$ for all ξ, then $\mu \vdash \pi :_\Gamma \sigma$ implies $\mu \vdash \pi :_\Delta \sigma$.*

(2) *(generation) A filter containing every type given by one rule of the cell $\mu(\pi)$ in Γ, from observations of its components, contains every type of π in Γ.*

Both are proved by induction on derivations. Each inversion below is (2) with a filter chosen for the purpose, and Lemma 2.2. For example, a cell (CONS $\pi_1\ \pi_2$) has type $\sigma \times \tau$ iff π_1 has type σ and π_2 type τ; it has no type $\ulcorner a \urcorner$ and no arrow type. For a FUNARG cell the filter "the types derived from one choice of types for θ" is closed under meets because two choices have a meet, again a choice, to which Lemma 3.2(1) moves both derivations; so every type of (FUNARG $\pi'\ \theta$) comes from a single choice of types for θ.

4. The rules preserve the meaning

Theorem 4.1. *If $\mu \Rightarrow \mu'$ by one of the rules, then for every non-free cell π of μ, every Γ and every σ: $\mu \vdash \pi :_\Gamma \sigma \iff \mu' \vdash \pi :_\Gamma \sigma$.*

The common part. Consider first a rewrite of one cell π_0. Define $\mu \Rrightarrow \nu$ at π_0: every type that one rule of the cell $\mu(\pi_0)$ derives from ν-observations of its components is a ν-type of π_0. If $\mu \Rrightarrow \nu$ at π_0 and μ, ν agree elsewhere, then every μ-observation is a ν-observation: by induction on the derivation, copying every step that does not use the cell π_0 and using the hypothesis for the steps that do. So each direction of each rule is one local lemma. An allocation keeps the types of the old cells, both ways: forwards the fresh cell has no rule in μ; backwards, a rule at an old cell has its premises at its components, which are old since the fresh cell occurs in no cell of μ, or (for LABEL) at the cell itself.

The forward directions of $[A]$, $[E]$, $[I]$, PLUS and $[Fb]$ use Lemma 2.2 to discard impossible cases: $\ulcorner a \urcorner \le \ulcorner b \urcorner$ gives $a = b$, $\ulcorner a \urcorner$ is below no pair type, and a CONS cell has no atom type.

The local lemmas. We give three; the others are similar.

- $[C]$: $\mu(\pi_0) = (\mathsf{CAR}\ \pi_1)$, $\mu(\pi_1) = (\mathsf{CONS}\ \pi_2\ \pi_3)$, $\mu' = \mu[\mu(\pi_2)/\pi_0]$. A type σ of π_0 by the CAR rule comes from a type $\sigma \times \tau$ of π_1, hence (generation) from a type σ of π_2, whose last rule is the rule of the cell $\mu(\pi_2) = \mu'(\pi_0)$. Conversely, π_0 and π_2 hold the same cell in μ', so they have the same types, by induction on derivations (a LABEL premise at π_0 is read at π_2); and π_3 has type ω, so π_0 has type σ by CAR of $\sigma \times \omega$. The other copying rules ($[I]$, $[Fa]$) are treated alike.
- $[G]$: $\mu(\pi_0) = (\pi_f\ \pi_2)$, $\mu(\pi_f) = (\mathsf{FUNARG}\ \pi_\lambda\ \theta)$, $\mu(\pi_\lambda) = (\mathsf{LAMBDA}\ \xi\ \pi_1)$, $\mu' = \mu[(\mathsf{FUNARG}\ \pi_1\ \xi\ \pi_2\ \theta)/\pi_0]$. A type τ of π_0 by application comes from $\sigma \to \tau$ at π_f and σ at π_2; by generation, from a single choice $\vec{\sigma}$ of types for θ and, by Lemma 2.2(3),

from $\pi_1 : \tau$ in $\Gamma[\theta : \vec{\sigma}][\xi : \sigma]$, the environment of the new FUNARG cell with the choice $\vec{\sigma}$ updated at ξ. Conversely, a derivation at the new cell gives $\pi_2 : \sigma$ and a choice for θ; the choice for the old FUNARG cell must give ω to ξ, since its binding of ξ is shadowed by the new one.

- $[Fa]$: $\mu(\pi_0) = (\mathsf{FUNARG}\ \pi'\ \theta)$, $\mu(\pi') = \xi$, i the smallest index with $\xi_i = \xi$, $\mu' = \mu[\mu(\pi_i)/\pi_0]$. A type of π_0 is above $\Gamma[\theta : \vec{\sigma}](\xi) = \sigma_i$, a type of π_i; and conversely.
- $[Fc]$, $[Fd]$: after the allocation of fresh cells π'_j holding $(\mathsf{FUNARG}\ \pi_j\ \theta)$, the cell $(\mathsf{FUNARG}\ (\pi_1\ \pi_2)\ \theta)$ becomes $(\pi'_1\ \pi'_2)$. Forwards, the types chosen for θ at π_0 serve for both new cells. Backwards, the two new cells may choose different types for θ; their meet is a choice of types for θ, and Lemma 3.2(1) moves both subderivations to it.

The circular rule $[L]$. Here $\mu(\pi_0) = (\mathsf{FUNARG}\ \pi_\ell\ \theta)$, $\mu(\pi_\ell) = (\mathsf{LABEL}\ \xi\ \pi_1)$ and $\nu = \mu[(\mathsf{FUNARG}\ \pi_1\ \xi\ \pi_0\ \theta)/\pi_0]$.

($\mu \Rightarrow \nu$.) By induction on the derivation of $\mu \vdash \pi_\ell :_{\Gamma[\theta:\vec{\sigma}]} \tau$, with $\vec{\sigma}$ types of θ in ν, using the induction hypothesis on the self-premise of the LABEL rule: that rule uses $\pi_1 : \tau$ in $\Gamma[\theta : \vec{\sigma}][\xi : \sigma]$ and a strictly smaller derivation of $\pi_\ell : \sigma$, which by induction gives $\pi_0 : \sigma$ in ν; these are exactly the premises of the FUNARG rule of $\nu(\pi_0)$, the binding $\xi\pi_0$ being first. Each unfolding of the label becomes one step through the circular binding: this single induction replaces the paper's two nested approximations V_i and Y_i.

($\nu \Rightarrow \mu$.) By the common part, it suffices that a type τ given at π_0 by the FUNARG rule of $\nu(\pi_0)$, from μ-observations, is a μ-type of π_0. That rule gives a choice $\vec{\sigma}$ for θ, a type σ of π_0 for ξ, and $\pi_1 : \tau$ in $\Gamma[\theta : \vec{\sigma}][\xi : \sigma]$. In μ, by generation, $\pi_0 : \sigma$ comes from a choice $\vec{\sigma'}$ for θ and $\pi_\ell : \sigma$ in $\Gamma[\theta : \vec{\sigma'}]$. Their meet $\vec{\delta}$ is a choice for θ, and by Lemma 3.2(1) both derivations hold with $\vec{\delta}$; the LABEL rule then gives $\pi_\ell : \tau$ in $\Gamma[\theta : \vec{\delta}]$, hence $\pi_0 : \tau$. The meet always exists, inconsistent or not: no consistency of the observations of a cell is needed.

5. Partial correctness

Corollary 5.1. *Let* $\mathsf{Eval}(\pi_0, \mu)$ *halt with* μ', π_0 *non-free in* μ. *Then* π_0 *has the same types in* μ *and* μ', *in every environment, and: if* $\mu'(\pi_0) = (\mathsf{QUOTE}\ a)$, *the types of* π_0 *are exactly those above* $\ulcorner a \urcorner$: *its value is the atom* a; *if* $\mu'(\pi_0)$ *is a* CONS, π_0 *has type* $\omega \times \omega$ *and no atom or arrow type: its value is a pair; if* $\mu'(\pi_0) = (\mathsf{FUNARG}\ (\mathsf{LAMBDA} \ldots) \ldots)$, *it has type* $\omega \to \omega$ *and no atom or pair type: its value is a function.*

Proof. Eval performs a finite sequence of allocations and rule applications, so Theorem 4.1 applies at each step; π_0 stays non-free, since every rewrite writes a non-free cell (the copies are of evaluated cells). The three cases are Lemma 3.2(2) at the final cell, with Lemma 2.2. □

As in the paper, the same argument shows that the final form does not depend on the order of the rewrites. No consistency of the memory is used: the meaning of π_0 is shown equal to a principal filter of a consistent type. Example 4 of the paper (`integers`, evaluating to 1) is the instance where $\mu'(\pi_0) = (\mathsf{QUOTE}\ 1)$.

6. Infinite elements: the semantics of Henderson and Morris

The preceding sections use only the compact elements. We now add the infinite elements, as filters of types, and recover the paper's statements literally: its semantic function is the least solution of its own equations, and each rule leaves the semantic memory unchanged. This is the construction of the elements as filters of compact elements [2, 5, 3], as in the filter model [4], here for the domain with separated sum.

6.1. **The domain.** A *filter* is a set x of types with $\omega \in x$, closed under $\wedge$ and upward closed for $\leq$. Let $\mathcal{D}$ be the set of filters, ordered by inclusion. It is a complete lattice: the intersection of filters is a filter, and so is a directed union. Its least element is $\bot = \uparrow\omega = \{\sigma \mid \omega \leq \sigma\}$; its greatest element is the set of all types. The *compact* elements are the principal filters $\uparrow\sigma$, and every filter is the directed union of the $\uparrow\sigma$, $\sigma \in x$. We keep all the filters, the inconsistent ones included: $\mathcal{D}$ is a lattice, as the paper's domain is ("$\bot$ being the bottom of the whole lattice"), and nothing requires consistency. Two filters are equal when they have the same elements.

For a set A of types, the filter *generated* by A is $\langle A\rangle = \{\tau \mid \omega \leq \tau$, or $\sigma_1 \wedge \cdots \wedge \sigma_n \leq \tau$ for some $\sigma_1, \ldots, \sigma_n \in A$, $n \geq 1\}$. The operations of the paper become, for $x, y, z \in \mathcal{D}$:

$$\langle x, y\rangle = \langle\{\sigma \times \tau \mid \sigma \in x,\ \tau \in y\}\rangle, \qquad x.1 = \{\sigma \mid \omega \leq \sigma \text{ or } \sigma \times \omega \in x\}, \quad x.2 \text{ likewise},$$
$$x \cdot y = \{\tau \mid \omega \leq \tau \text{ or } \sigma \to \tau \in x \text{ for some } \sigma \in y\}, \quad \mathsf{atom}(x) = \langle\{\ulcorner \mathsf{T} \urcorner \mid \mathsf{atom} \in x\} \cup \{\ulcorner \mathsf{NIL} \urcorner \mid \omega \times \omega \in x\}\rangle,$$

$\mathsf{eq}(x, y)$ is generated by $\ulcorner \mathsf{T} \urcorner$ if $\ulcorner a \urcorner \in x$ and $\ulcorner a \urcorner \in y$ for some atom a, and by $\ulcorner \mathsf{NIL} \urcorner$ if $\ulcorner a \urcorner \in x$ and $\ulcorner b \urcorner \in y$ with $a \neq b$; $\mathsf{if}(x, y, z)$ is generated by y if $\ulcorner \mathsf{T} \urcorner \in x$ and by z if $\ulcorner \mathsf{NIL} \urcorner \in x$. Each is a filter (for $x \cdot y$ by the two arrow rules of Definition 2.1), and each is monotone in its arguments. The summand $\omega \leq \tau$ is the separated sum: $\bot \cdot y = \bot$, but $\langle \bot, \bot\rangle = \uparrow(\omega \times \omega) \neq \bot$.

A monotone $g : \mathcal{D} \to \mathcal{D}$ is *continuous* if $g(x) = \bigcup_{\sigma \in x} g(\uparrow\sigma)$; it is then determined by its *trace*, the relation $\tau \in g(\uparrow\sigma)$ between types. For such g put

$$\Lambda g = \langle\{\sigma \to \tau \mid \tau \in g(\uparrow\sigma)\}\rangle,$$

and let Yg be the least set of types containing ω, closed under $\wedge$ and $\leq$, and such that $\sigma \in Yg$ and $\tau \in g(\uparrow\sigma)$ give $\tau \in Yg$. Both depend only on the trace of g. Note $\omega \to \omega \in \Lambda g$, so $\Lambda g \neq \bot$: "(LAMBDA x $\bot$)" and $\bot$ are distinct, as the paper requires.

Lemma 6.1 (laws)**.** *For $x, y, z \in \mathcal{D}$, atoms a, b, and g continuous:*

1. $\langle x, y\rangle.1 = x$ *and* $\langle x, y\rangle.2 = y$*;*
2. *(β)* $(\Lambda g) \cdot y = g(y)$*;*
3. $Yg = g(Yg)$*, and* $g(x) \subseteq x$ *implies* $Yg \subseteq x$*;*
4. $\mathsf{atom}(\uparrow\ulcorner a \urcorner) = \uparrow\ulcorner \mathsf{T} \urcorner$, $\mathsf{atom}\langle x, y\rangle = \uparrow\ulcorner \mathsf{NIL} \urcorner$*;* $\mathsf{eq}(\uparrow\ulcorner a \urcorner, \uparrow\ulcorner a \urcorner) = \uparrow\ulcorner \mathsf{T} \urcorner$, $\mathsf{eq}(\uparrow\ulcorner a \urcorner, \uparrow\ulcorner b \urcorner) = \uparrow\ulcorner \mathsf{NIL} \urcorner$ *for $a \neq b$;* $\mathsf{if}(\uparrow\ulcorner \mathsf{T} \urcorner, y, z) = y$, $\mathsf{if}(\uparrow\ulcorner \mathsf{NIL} \urcorner, y, z) = z$*;*
5. $\uparrow\ulcorner a \urcorner$, $\langle x, y\rangle$ *and* Λg *are pairwise distinct and distinct from $\bot$: they contain respectively* $\ulcorner a \urcorner$, $\omega \times \omega$, $\omega \to \omega$*, and no other of these three types.*

Proof. All but (3) are Lemma 2.2 read on generated filters. For (1): if $\sigma \times \omega \in \langle x, y\rangle$ and $\omega \not\leq \sigma$, then $\bigwedge(\sigma_i \times \tau_i) \leq \sigma \times \omega$ with $\sigma_i \in x$, so $\bigwedge \sigma_i \leq \sigma$ and $\sigma \in x$. For (2): let $\sigma \to \tau \in \Lambda g$ with $\sigma \in y$; then $\bigwedge_k(\sigma_k \to \tau_k) \leq \sigma \to \tau$ with $\tau_k \in g(\uparrow\sigma_k)$ ($\omega \leq \sigma \to \tau$ is impossible), and Lemma 2.2(3) gives a selection with $\sigma \leq \sigma_k$ and $\bigwedge \tau_k \leq \tau$; since $\uparrow\sigma_k \subseteq \uparrow\sigma \subseteq y$, monotonicity gives $\tau \in g(y)$. Conversely $\tau \in g(y)$ is in some $g(\uparrow\sigma)$, $\sigma \in y$, and $\sigma \to \tau \in \Lambda g$. For (3): if $g(x) \subseteq x$, then $Yg \subseteq x$ by induction on the generation of Yg, since $\sigma \in x$ and $\tau \in g(\uparrow\sigma)$ give $\tau \in g(x) \subseteq x$. Next $Yg \subseteq g(Yg)$ by the same induction, $\uparrow\sigma \subseteq Yg$ giving $g(\uparrow\sigma) \subseteq g(Yg)$; and $g(Yg) \subseteq Yg$ by continuity and the last closure rule. □

The least fixed point is thus obtained by an inductive generation, not as the limit of the iterates $g^n(\bot)$; (3) is fixed-point induction (Park's principle [7]).

6.2. **Semantic memories.** An *environment* ρ maps variables to filters. A type environment Γ (Section 3) is *in* ρ when $\Gamma(\xi) \in \rho(\xi)$ for all ξ; write $\uparrow\Gamma$ for the environment $\xi \mapsto \uparrow\Gamma(\xi)$. A function F from environments to filters is continuous if it is monotone and $F(\rho) = \bigcup_{\Gamma \in \rho} F(\uparrow\Gamma)$.

A *semantic memory* S maps each address to a continuous function from environments to filters. Following the paper, for a cell e let $\mathcal{U}(S)(e)\rho$ be:

$\rho(\xi)$	$e = \xi$
$\uparrow\ulcorner a \urcorner$	$e = (\mathsf{QUOTE}\ a)$
$\langle S\pi_1\rho, S\pi_2\rho\rangle$	$e = (\mathsf{CONS}\ \pi_1\ \pi_2)$
$(S\pi'\rho).1, \quad (S\pi'\rho).2$	$e = (\mathsf{CAR}\ \pi'),\ (\mathsf{CDR}\ \pi')$
$\mathsf{atom}(S\pi'\rho),\ \mathsf{eq}(S\pi_1\rho, S\pi_2\rho),\ \mathsf{if}(S\pi_0\rho, S\pi_1\rho, S\pi_2\rho)$	$e = (\mathsf{ATOM}\ \pi'),\ (\mathsf{EQ}\ \pi_1\ \pi_2),\ (\mathsf{IF}\ \pi_0\ \pi_1\ \pi_2)$
$S\pi_0\rho \cdot S\pi_1\rho$	$e = (\pi_0\ \pi_1)$
$\Lambda(x \mapsto S\pi'\rho[x/\xi])$	$e = (\mathsf{LAMBDA}\ \xi\ \pi')$
$Y(x \mapsto S\pi'\rho[x/\xi])$	$e = (\mathsf{LABEL}\ \xi\ \pi')$
$S\pi'\rho[S\theta\rho/\theta]$	$e = (\mathsf{FUNARG}\ \pi'\ \theta)$
$\bot$	π free

where $\rho[S\theta\rho/\theta]$ binds each ξ_i of $\theta = \xi_1\pi_1 \ldots \xi_n\pi_n$ to $S\pi_i\rho$, the smallest i winning. Each clause is continuous in ρ (for Y, by induction on its generation). The paper defines its semantic function $\mathcal{V}$ as the least solution of $\mathcal{V}\mu\pi = \mathcal{U}(\mathcal{V}\mu)(\mu\pi)$. We take the observations of Section 3 as the definition:

$$\mathcal{V}\mu\pi\rho = \{\sigma \mid \mu \vdash \pi :_\Gamma \sigma \text{ for some } \Gamma \text{ in } \rho\}.$$

Theorem 6.2. (1) *$\mathcal{V}\mu\pi\rho$ is a filter, continuous in ρ, and $\sigma \in \mathcal{V}\mu\pi(\uparrow\Gamma)$ iff $\mu \vdash \pi :_\Gamma \sigma$.*
(2) *$\mathcal{V}\mu\pi = \mathcal{U}(\mathcal{V}\mu)(\mu\pi)$ for every π.*
(3) *If S is a semantic memory with $\mathcal{U}(S)(\mu\pi) \subseteq S\pi$ for all π, then $\mathcal{V}\mu\pi \subseteq S\pi$ for all π.*

Proof. (1) ω is a type of π in the empty environment. If Γ_1 and Γ_2 are in ρ, so is their pointwise meet, and Lemma 3.2(1) moves both derivations to it: this gives the meet rule and continuity. The last claim is Lemma 3.2(1).

($\supseteq$ in (2)) Every generator of $\mathcal{U}(\mathcal{V}\mu)(\mu\pi)\rho$ is the conclusion of the rule of the cell $\mu\pi$, in an environment in ρ obtained as the meet of the environments of the premises. For LAMBDA: if $\tau \in \mathcal{V}\mu\pi'(\rho[\uparrow\sigma/\xi])$, by Γ', then $\Gamma = \Gamma'[\xi : \omega]$ is in ρ and $\Gamma[\xi : \sigma]$ is below Γ', so $\pi : \sigma \to \tau$ in Γ. For LABEL use Lemma 6.1(3) with $x = \mathcal{V}\mu\pi\rho$: by continuity, an element of $\mathcal{V}\mu\pi'(\rho[x/\xi])$ is in some $\mathcal{V}\mu\pi'(\rho[\uparrow\sigma/\xi])$ with $\sigma \in x$, and the LABEL rule applies. For FUNARG the environment of the premises meets the environments given for the π_i; this is where the meet of the choices of types for θ (Section 4) now happens, inside the filters.

($\subseteq$ in (2), and (3)) Let S satisfy $\mathcal{U}(S)(\mu\pi) \subseteq S\pi$ for all π. By induction on derivations, $\mu \vdash \pi :_\Gamma \sigma$ implies $\sigma \in \mathcal{U}(S)(\mu\pi)\rho$ for every ρ containing Γ: the premises at a component π_i give, by induction, types in $\mathcal{U}(S)(\mu\pi_i) \subseteq S\pi_i$, and the rule is the corresponding generator. For LABEL the premise $\mu \vdash \pi :_\Gamma \sigma$ at the cell itself gives, by induction, $\sigma \in \mathcal{U}(S)(\mu\pi)\rho = Y(\ldots)$, and with $\tau \in S\pi'(\rho[\uparrow\sigma/\xi])$ the closure rule of Y gives $\tau \in Y(\ldots)$. Hence $\mathcal{V}\mu\pi \subseteq \mathcal{U}(S)(\mu\pi) \subseteq S\pi$. Taking $S = \mathcal{V}\mu$, a prefixed point by the first part, gives $\subseteq$ in (2). □

So $\mathcal{V}\mu$ is the least fixed point of $S \mapsto \mathcal{U}(S)(\mu\,\cdot)$, that is, the paper's $\mathcal{V}\mu = \lim_i \mathcal{V}_i\mu$; the approximations are not needed. By (1), Theorem 4.1 is the statement $\mathcal{V}\mu\pi = \mathcal{V}\mu'\pi$ restricted to the environments $\uparrow\Gamma$, and continuity gives the converse. We now prove the equalities directly, as the paper does.

6.3. **The rules leave the semantic memory unchanged.**

Lemma 6.3 (comparison)**.** *Let $\nu = \mu[e/\pi_0]$. If $\mathcal{U}(\mathcal{V}\nu)(\mu\pi_0) \subseteq \mathcal{V}\nu\pi_0$, then $\mathcal{V}\mu \subseteq \mathcal{V}\nu$.*

Proof. For $\pi \neq \pi_0$, $\mathcal{U}(\mathcal{V}\nu)(\mu\pi) = \mathcal{U}(\mathcal{V}\nu)(\nu\pi) = \mathcal{V}\nu\pi$ by Theorem 6.2(2); so $\mathcal{V}\nu$ satisfies the hypothesis of Theorem 6.2(3) for μ. □

So for each rule two inclusions at π_0 are to be checked, and for all rules but $[L]$ both follow from one equation: if $\mathcal{U}(S)(\mu\pi_0) = \mathcal{U}(S)(\nu\pi_0)$ for every semantic memory S satisfying $S\pi = \mathcal{U}(S)(\mu\pi)$ at the cells $\pi \neq \pi_0$, then, taking $S = \mathcal{V}\nu$ and $S = \mathcal{V}\mu$, $\mathcal{V}\mu = \mathcal{V}\nu$. Note that $\mathcal{U}(S)(e)$ does not mention the address of e; this is why the copying rules need no care about LABEL cells here. The equations, with $\rho' = \rho[S\theta\rho/\theta]$:

- $[C]$: $\mathcal{U}(S)(\mathsf{CAR}\,\pi_1)\rho = (S\pi_1\rho).1 = \langle S\pi_2\rho, S\pi_3\rho\rangle.1 = S\pi_2\rho = \mathcal{U}(S)(\mu\pi_2)\rho$, by Lemma 6.1(1). (If $\pi_2 = \pi_0$ then $\nu = \mu$.)
- $[A]$, $[E]$, $[I]$: Lemma 6.1(4), in the same way.
- $[Fa]$: $\mathcal{U}(S)(\mathsf{FUNARG}\,\pi'\,\theta)\rho = S\pi'\rho' = \rho'(\xi) = S\pi_i\rho = \mathcal{U}(S)(\mu\pi_i)\rho$; $[Fb]$: $S\pi'\rho' = \uparrow^{\ulcorner} a^{\urcorner}$.
- $[G]$: $\mathcal{U}(S)(\pi_f\ \pi_2)\rho = S\pi_f\rho \cdot S\pi_2\rho = \Lambda(x \mapsto S\pi_1\rho'[x/\xi]) \cdot S\pi_2\rho = S\pi_1\rho'[S\pi_2\rho/\xi] = \mathcal{U}(S)(\mathsf{FUNARG}\ \pi_1\ \xi\,\pi_2\,\theta)\rho$, by β (Lemma 6.1(2)).

The circular rule $[L]$, with $\mu\pi_0 = (\mathsf{FUNARG}\,\pi_\ell\,\theta)$, $\mu\pi_\ell = (\mathsf{LABEL}\,\xi\,\pi_1)$, $\nu\pi_0 = (\mathsf{FUNARG}\,\pi_1\,\xi\,\pi_0\,\theta)$. Write $g_S(x) = S\pi_1\rho'[x/\xi]$. Then $\mathcal{U}(S)(\mu\pi_0)\rho = S\pi_\ell\rho' = Yg_S$ and $\mathcal{U}(S)(\nu\pi_0)\rho = g_S(S\pi_0\rho)$.

- $(\mathcal{V}\mu \subseteq \mathcal{V}\nu)$ For $S = \mathcal{V}\nu$: $g_S(\mathcal{V}\nu\pi_0\rho) = \mathcal{U}(\mathcal{V}\nu)(\nu\pi_0)\rho = \mathcal{V}\nu\pi_0\rho$, so $Yg_S \subseteq \mathcal{V}\nu\pi_0\rho$ by Lemma 6.1(3).
- $(\mathcal{V}\nu \subseteq \mathcal{V}\mu)$ For $S = \mathcal{V}\mu$: $\mathcal{V}\mu\pi_0\rho = Yg_S$, so $\mathcal{U}(\mathcal{V}\mu)(\nu\pi_0)\rho = g_S(Yg_S) = Yg_S = \mathcal{V}\mu\pi_0\rho$.

The paper proves the second inclusion by cutting the loop of Y and the loop of $\mathcal{V}$ together (approximations $\mathcal{V}_i$ and Y_i); with both characterised as least fixed points, each inclusion is one application of fixed-point induction.

The allocating rules $[Fc]$, $[Fd]$: $\mu\pi_0 = (\mathsf{FUNARG}\,\pi_a\,\theta)$ with $\mu\pi_a$ a primitive form $p(\pi_1, \ldots, \pi_k)$ (an application or a primitive). The rule first allocates cells π'_j holding $(\mathsf{FUNARG}\ \pi_j\ \theta)$, then rewrites π_0 to $p(\pi'_1, \ldots, \pi'_k)$. The rewrite is a one-cell rule: with P the operation of p, $\mathcal{U}(S)(\mu\pi_0)\rho = S\pi_a\rho' = P(S\pi_1\rho', \ldots) = P(S\pi'_1\rho, \ldots)$, since $S\pi'_j\rho = S\pi_j\rho'$. An allocation $\nu = \mu[(\mathsf{FUNARG}\ \pi_j\ \theta)/\pi']$ takes a free cell π' occurring in no cell of μ (as in the paper, no non-free cell points at a free one). Then $\mathcal{V}\mu \subseteq \mathcal{V}\nu$ by Lemma 6.3, since $\mathcal{U}(\mathcal{V}\nu)(\mu\pi') = \bot$. Conversely, let S be $\mathcal{V}\mu$ except at π', where it is the filter of all types. No cell of μ mentions π', so $\mathcal{U}(S)(\nu\pi) = \mathcal{U}(\mathcal{V}\mu)(\mu\pi) \subseteq S\pi$ for $\pi \neq \pi'$, and $\mathcal{U}(S)(\nu\pi') \subseteq S\pi'$ trivially. Hence $\mathcal{V}\nu \subseteq S$ by Theorem 6.2(3), and $\mathcal{V}\nu\pi = \mathcal{V}\mu\pi$ at the non-free cells of μ.

Theorem 6.4. *If $\mu \Rightarrow \nu$ by one of the rules, then $\mathcal{V}\mu\pi = \mathcal{V}\nu\pi$ for every non-free cell π of μ. If* $\mathsf{Eval}(\pi_0, \mu)$ *halts, then $\mathcal{V}\mu\pi_0\bot$ is $\uparrow^{\ulcorner} a^{\urcorner}$, a pair $\langle x, y\rangle$ or a function Λg, according as it halts with* $(\mathsf{QUOTE}\ a)$*, a* CONS *or a* FUNARG *of a* LAMBDA *at π_0. In particular, if $\mathcal{V}\mu\pi_0\bot = \bot$ then* $\mathsf{Eval}(\pi_0, \mu)$ *does not halt, and if $\mathcal{V}\mu\pi_0\bot = \uparrow^{\ulcorner} a^{\urcorner}$ and it halts, it halts with* $(\mathsf{QUOTE}\ a)$*.*

These are the paper's four correctness clauses; the last two sentences are Lemma 6.1(5), which rests on the separated sum.

7. Relation to the finite-element presentation

An earlier formalisation, in Agda, presents the compact elements as finite elements, ordered by an algorithm. Up to the equivalence $\sigma \leq \tau \leq \sigma$, the consistent types are the finite elements: ω is $\bot$, $\ulcorner a \urcorner$ an atom, $\sigma \times \tau$ a pair and $\bigwedge(u_i \to v_i)$ the step function $\{(u_i, v_i)\}$. Subtyping is the *reverse* of the information order: $u \sqsubseteq v$ iff $v \leq u$, so ω, the top type, is the least element $\bot$, and the meet of types is the join of the corresponding elements, when they are consistent. What changes is the presentation of the order: by inference rules instead of an algorithm computing $h(u)$. This removes the stage stratification of the order and its stability lemmas, the duplicate (propositional and decidable) orders, the consistency proof of observations, and the "shape-directed" inversion forced by the closure rules (here: Lemma 3.2(2)). The order

algorithm, if wanted for computation, can be added afterwards as a decision procedure proved equivalent to $\leq$.

8. Formalisation in Unit

Types are one ordinary inductive type. Subtyping $\leq$, its syntax-directed form $\leq^s$ (with a family of selections of arrow components), and the observation relation are indexed inductive families [8], with all recursive premises positive; none of them is mutual or nested. The rule instances of a cell are separated from their premises: the premises form a function field indexed by a type of premises, which gives the FUNARG rule one premise per binding. The memory is a parameter (a function from addresses to cells), the environment a list of types. The evaluator is a relation (big-step, as Eval), each case restating the cells of its rewrite in the memory reached. Sections 2–5 are formalised in `domain/lazy`, with no parameters; for Example 4 the observation derivation, not the paper's run of the evaluator.

The infinite elements of Section 6 need function extensionality and propositional extensionality (both consequences of univalence) to state equalities of filters and of semantic memories, and propositional truncation for "for some Γ in ρ". They are parameters of the development, which the finite part above does not use. A filter is a proposition-valued predicate on types with the three closure properties; the filters form a type one universe up. Continuous functions are given by their traces, relations between types, so that Λ and Y are inductive filters in the universe of the types and no resizing is needed. This part is formalised in `domain/ideal` (filters and operations, Lemma 6.1, Theorem 6.2, Lemma 6.3 and Theorem 6.4); its proof of Theorem 6.4 uses the observations only through Theorem 6.2, not Theorem 4.1. Of Theorem 6.2(1), the inclusion of the observations in $\mathcal{V}\mu\pi(\uparrow\Gamma)$ is formalised; the converse, not used, would need a choice over the variables of an environment.

References

[1] P. Henderson and J. H. Morris, A lazy evaluator. *Proc. 3rd ACM Symposium on Principles of Programming Languages*, 1976, 95–103.
[2] P. Martin-Löf, The domain interpretation of type theory. Lecture notes, Workshop on Semantics of Programming Languages (K. Karlsson, K. Petersson, eds.), Chalmers University of Technology and University of Göteborg, August 1983.
[3] S. Abramsky, Domain theory in logical form. *Annals of Pure and Applied Logic* 51 (1991), 1–77; LICS 1987.
[4] H. Barendregt, M. Coppo and M. Dezani-Ciancaglini, A filter lambda model and the completeness of type assignment. *Journal of Symbolic Logic* 48 (1983), 931–940.
[5] D. Scott, Domains for denotational semantics. *ICALP 1982*, LNCS 140, 577–613.
[6] J. C. Reynolds, Notes on a lattice-theoretic approach to the theory of computation. Lecture notes, Syracuse University, 1971.
[7] D. Park, Fixpoint induction and proofs of program properties. *Machine Intelligence* 5 (1969), 59–78.
[8] Th. Coquand and C. Paulin-Mohring, Inductively defined types. *COLOG-88*, LNCS 417 (1990), 50–66.